សប្តាហ៍ទី 2 · វិធាន និងការសម្រេចចិត្ត

Programming for Information Systems

តើនរណាជាអ្នកជ្រើសរើសវិធាន ហើយតើកម្មវិធីកុំព្យូទ័រអនុវត្តវាដោយរបៀបណា?

ចាប់ផ្តើមថ្នាក់ · ចែករំលែកកិច្ចការស្រាវជ្រាវទី 1

ជាក្រុមបីនាក់ ចែករំលែកប្រព័ន្ធព័ត៌មានដែលអ្នកបានស្រាវជ្រាវ។

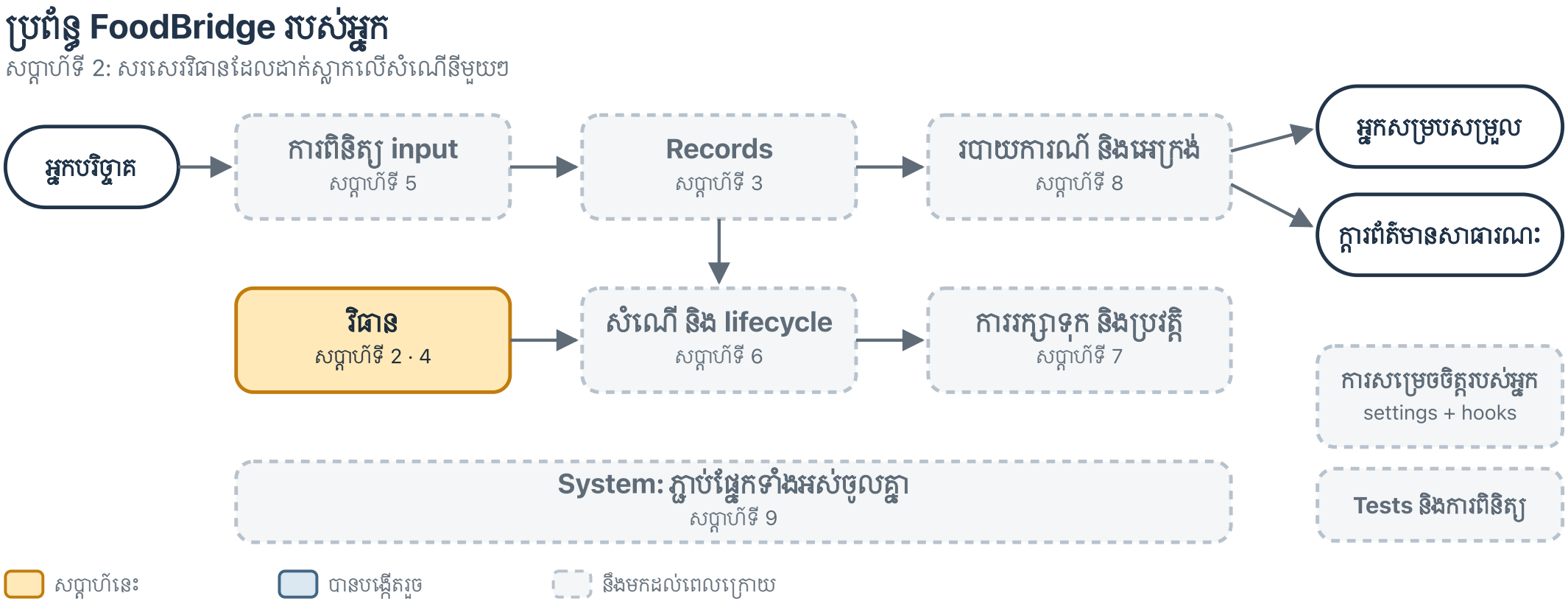
1. តើនរណាជាអ្នកប្រើប្រាស់វា ហើយតើវាផ្តុកទិន្នន័យអ្វីខ្លះ?
2. តើអ្នកបានរកឃើញច្បាប់ (rule) អ្វីខ្លះ? តើវាត្រូវបានសរសេរទុក ឬមានតែនៅក្នុងគំនិតរបស់មនុស្ស?
3. តើមានអ្វីអាចនឹងខុសឆ្គង ហើយតើនរណានឹងរងផលប៉ះពាល់?

> 📝 កត់ត្រាគំនិតមួយពីមិត្តរួមថ្នាក់ ដែលអ្នកចង់ចងចាំ។

សប្តាហ៍នេះ

សំណួរ	តើប្រព័ន្ធធ្វើការសម្រេចចិត្តយ៉ាងដូចម្តេច ហើយតើនរណាជាអ្នកជ្រើសរើសវិធាននេះ?
Python	ការប្រៀបធៀប, `True` / `False`, `and` / `or` / `not`, `if` / `elif` / `else`, function តូចមួយដំបូង
គោលការណ៍ទី 2	វិធានត្រូវច្បាស់លាស់: ដាក់ឈ្មោះឱ្យតួលេខ គ្រប់ដណ្តប់គ្រប់ករណី និងសាកល្បងនៅចំណុចព្រំដែន
អ្នកបង្កើត	`pickup_label` និង `urgency`

ប្រព័ន្ធរបស់អ្នក ត្រឹមសប្តាហ៍ទី 10



គំនិតសំខាន់: វិធានអាជីវកម្ម (business rule)

វិធានអាជីវកម្ម គឺជាការសម្រេចចិត្តដែលស្ថាប័នមួយបង្កើតឡើង អំពីរបៀបដែលការងារត្រូវអនុវត្ត។

- *"ពិន្ទុប្រឡងចាប់ពី 50 ឡើងទៅគឺជាប់។"*
- *"ក្មេងអាយុក្រោម 6 ឆ្នាំធ្វើដំណើរដោយឥតគិតថ្លៃ។"*

> **អ្នកសម្របសម្រួល:** "មិនទាន់អនុម័ត → **hold**។ បានអនុម័ត ហើយមាន **20 ចំណែក ឬច្រើនជាងនេះ** → **team**។ បើមិនដូច្នោះទេ → **standard**។"

- មនុស្សជាអ្នកជ្រើសរើសវិធាន។ កម្មវិធីកុំព្យូទ័រអនុវត្តវាដដែលៗដូចគ្នារាល់ដង។
- វិធានដែលខុសនឹងត្រូវបានអនុវត្តយ៉ាងខុសឆ្គងរាល់ដង ដូច្នេះវិធានត្រូវតែ **សរសេរឱ្យបានច្បាស់លាស់** និង **សាកល្បងនៅចំណុចព្រំដែន**។

មូលដ្ឋាន Python

របៀបដែលកម្មវិធីកុំព្យូទ័រធ្វើការសម្រេចចិត្ត

ការប្រៀបធៀបផ្តល់ `True` ឬ `False`

```
portions = 20
print(portions >= 20)        # True
print(portions > 20)         # False
print(portions == 20)        # True
print("bread" == "Bread")    # False
```

- `==` `!=` `<` `<=` `>` `>=`
- "20 ឬច្រើនជាងនេះ" គឺ `>= 20` ។ "ច្រើនជាង 20" គឺ `> 20` ។
- ការប្រៀបធៀបអត្ថបទ (text) ត្រូវតែដូចគ្នាពិតប្រាកដ ដូច្នេះអក្សរធំ-តូចគឺសំខាន់។

ការផ្សំលក្ខខណ្ឌ: `and` (និង), `or` (ឬ), `not` (មិន)

```
approved = True
portions = 25
print(approved and portions >= 20)        # ទាំងពីរត្រូវតែ True
print(not approved)                       # ផ្ទុយ
print(portions >= 20 or portions < 0)     # យ៉ាងហោចណាស់មួយ True
```

- `and`: គ្រប់ផ្នែកត្រូវតែ `True`។
- `or`: យ៉ាងហោចណាស់ផ្នែកមួយត្រូវតែ `True`។
- `not`: ប្តូរ `True` ទៅជា `False` ហើយផ្ទុយមកវិញ។

`if` និង `else`: ការជ្រើសរើសអ្វីដែលត្រូវធ្វើ

```
temperature = 31
if temperature >= 30:
    print("Hot: keep the food in the shade")
else:
    print("Normal temperature")
print("This line always runs")
```

- បន្ទាត់ដែលមាន `if` បញ្ចប់ដោយសញ្ញាចុចពីរ (`:`)។
- បន្ទាត់ខាងក្រោមត្រូវតែ **ចូលបន្ទាត់ (indent)** ចំនួន 4 space។
- ការចូលបន្ទាត់មិនមែនគ្រាន់តែជាការតុបតែងទេ: វាកំណត់ថាបន្ទាត់ណាជារបស់អ្វី។

`elif`: ត្រួតពិនិត្យតាមលំដាប់ លក្ខខណ្ឌដំបូងដែលពិតគឺឈ្នះ

```
mark = 72
if mark >= 80:
    grade = "A"
elif mark >= 65:
    grade = "B"
elif mark >= 50:
    grade = "C"
else:
    grade = "fail"
print(grade)
```

🤔 ពិន្ទុ 65 គត់ ទទួលបាននិទ្ទេសអ្វី? ចុះ 64 វិញ?

ដាក់ឈ្មោះឱ្យតម្លៃរបស់អ្នក

```
TEAM_PICKUP_MIN_PORTIONS = 20      # អ្នកសម្របសម្រួលជាអ្នកសម្រេច

if portions >= TEAM_PICKUP_MIN_PORTIONS:
    ...
```

- តម្លៃនៅក្នុងវិធានមួយ គឺជា **ការសម្រេចចិត្ត**។
- ឈ្មោះមួយជួយឱ្យមនុស្ស **ស្វែងរក** វា **យល់** វា និង **ផ្លាស់ប្តូរ** វានៅកន្លែងតែមួយ។
- ឈ្មោះជាអក្សរធំ គឺជា **តម្លៃថេរ (constant)**: កម្មវិធីកុំព្យូទ័រមិនប្តូរវាទេ ពេលវាកំពុងដំណើរការ។

Function ដំបូង: វេចខ្ជប់វិធានមួយ

```python
def is_hot(temperature):
    if temperature >= 30:
        return True
    return False

print(is_hot(31))    # True
print(is_hot(25))    # False
```

- `def` បង្ហាញកន្លែងចាប់ផ្តើម function; `temperature` គឺជាធាតុចូល (input) របស់វា។
- `return` បញ្ជូនចម្លើយត្រឡប់មកវិញ ហើយ **បញ្ឈប់** function ត្រឹមនោះ។
- ប្រព័ន្ធហៅវិធានដដែលនេះ សម្រាប់សំណើគ្រប់មួយ។

សាកល្បងនៅចំណុចព្រំដែន

```
def needs_team(portions):
    if portions > 20:       # វិធាន: "20 ឬច្រើនជាងនេះ"
        return True
    return False

print(19, needs_team(19))   # False  ✓
print(20, needs_team(20))   # False  ✗  គួរតែជា True
print(21, needs_team(21))   # True   ✓
```

> មានតែករណី **ព្រំដែន** ទេ ដែលបង្ហាញកំហុស (bug) នេះ។ ត្រូវសាកល្បងជានិច្ច: មួយខាងក្រោម **មួយចំព្រំដែន** និងមួយខាងលើ។

⚖️ ល្អ ឬមានហានិភ័យ?

Version A (ពីជំនួយការ AI)

```
def label(p, a):
    if p > 20 and a == True:
        return "team"
    elif a == True:
        return "standard"
```

Version B

```
TEAM_PICKUP_MIN_PORTIONS = 20

def pickup_label(portions, approved):
    if not approved:
        return "hold"
    if portions >= TEAM_PICKUP_MIN_PORTIONS:
        return "team"
    return "standard"
```

គោលការណ៍ទី 2 · វិធានច្បាស់លាស់

- **ដាក់ឈ្មោះឱ្យត្រូវលេខរបស់អ្នក**: តម្លៃថេរ (constant) មួយបង្ហាញការសម្រេចចិត្ត។
- **គ្របដណ្តប់គ្រប់ករណី**: គ្រប់ផ្លូវបញ្ចប់ដោយ `return`។
- **សាកល្បងនៅចំណុចព្រំដែន**: មួយខាងក្រោម មួយចំព្រំដែន និងមួយខាងលើ។

> ពេលអ្នកពិនិត្យកូដពី AI សូមសួរថា: *តើមានអ្វីកើតឡើងនៅចំណុចព្រំដែន ហើយពេលលក្ខខណ្ឌជា False?*

អ្នកបង្កើត: វិធានទៅយកទាំងពីរ

```
TEAM_PICKUP_MIN_PORTIONS = 20

def pickup_label(portions, approved):
    # មិនទាន់អនុម័ត → "hold"; ដល់ចំនួនអប្បបរមា → "team"
    # បើមិនដូច្នោះទេ → "standard"
    return "?"

bridge.check("pickup_label", pickup_label,
             {"TEAM_PICKUP_MIN_PORTIONS": TEAM_PICKUP_MIN_PORTIONS})
```

- `urgency(hours_left)`: ≤ `URGENT_HOURS` → "urgent", ≤ `SOON_HOURS` → "soon", ក្រៅពីនេះ "later"។
- ការពិនិត្យនេះធ្វើតេស្តលើករណីព្រំដែន **នៃតម្លៃរបស់អ្នកផ្ទាល់**។

ការដាក់បញ្ចូលវិធានទៅក្នុងប្រព័ន្ធ

```
MY_SETTINGS = {"TEAM_PICKUP_MIN_PORTIONS": TEAM_PICKUP_MIN_PORTIONS,
               "URGENT_HOURS": URGENT_HOURS, "SOON_HOURS": SOON_HOURS}
my_system = bridge.System(MY_SETTINGS,
                          hooks={"pickup_label": pickup_label, "urgency": urgency})
my_system.run_day(show=False)
print(my_system.report())
```

- **Hook** គឺជាកន្លែងដែលប្រព័ន្ធអនុញ្ញាតឱ្យអ្នកដាក់ function របស់អ្នកផ្ទាល់ចូល។
- ប្រសិនបើ function របស់អ្នកគាំង ប្រព័ន្ធនឹងបោះពុម្ពការព្រមាន ⚠️ ហើយប្រើវិធានដើម (default) របស់វា។

🧭 ការសម្រេចចិត្តរបស់អ្នក · ទំហំក្រុម និងភាពបន្ទាន់

- ប្តូរ `TEAM_PICKUP_MIN_PORTIONS`, `URGENT_HOURS`, `SOON_HOURS` (ឬរក្សាវាទុក ហើយពន្យល់ហេតុផល)។
- Run ម្តងទៀត: តេស្តនៅតែអាចឆ្លងរួច ហើយស្លាកប្តូរ។

កំណត់ត្រាការសម្រេចចិត្ត	
ជម្រើសរបស់ខ្ញុំ	team ចាប់ពី … ចំណែក; urgent ≤ … ម៉ោង; soon ≤ … ម៉ោង
ជម្រើសផ្សេង	…
ម្ចូលហេតុ	…
នរណាឯងផលប៉ះពាល់	…

កិច្ចការសប្តាហ៍នេះ

- ⭐ **មូលដ្ឋាន:** P1–P4, បញ្ចាំងក្នុង Version A, `pickup_label` និង `urgency` ដែល ✅ ទាំងអស់ ហើយដាក់បញ្ចូលទៅក្នុងប្រព័ន្ធ, កំណត់ត្រាការសម្រេចចិត្ត។
- ⭐⭐ **បន្ថែម:** S1 `send_team` ដែលមានលក្ខខណ្ឌពីរ; `explain_label` ដែលប្រាប់មូលហេតុ។
- 🔎 **កិច្ចការស្រាវជ្រាវទី 2 (5%):** វិធានពីជីវិតពិត ដែលសរសេរឱ្យច្បាស់លាស់ ជាមួយករណីធម្មតា ករណីព្រំដែន និងករណីចម្លែក។ ចែករំលែកវានៅពេលចាប់ផ្តើមថ្នាក់ សប្តាហ៍ទី 3។

សប្តាហ៍ក្រោយ · Records និងទិន្នន័យ

សំណើមួយគឺងាយស្រួល។ មួយថ្ងៃពេញមានសំណើច្រើន។ អ្នកនឹងរក្សា **record មួយសម្រាប់សំណើនីមួយៗ** ឬកសរុបវា ហើយរកសំណើណាមួយតាម **ID** របស់វា។